For example, navigation bars or footers that appear on every page only need to be defined once, and then the template engine helps you include them everywhere. Templates are not HTML-specific; they also get used for populating email content and anywhere else you wish to manage text, like reusable code. Figure 1.2 shows a view in more detail and its interactions with the ORM, database, and templating engine.

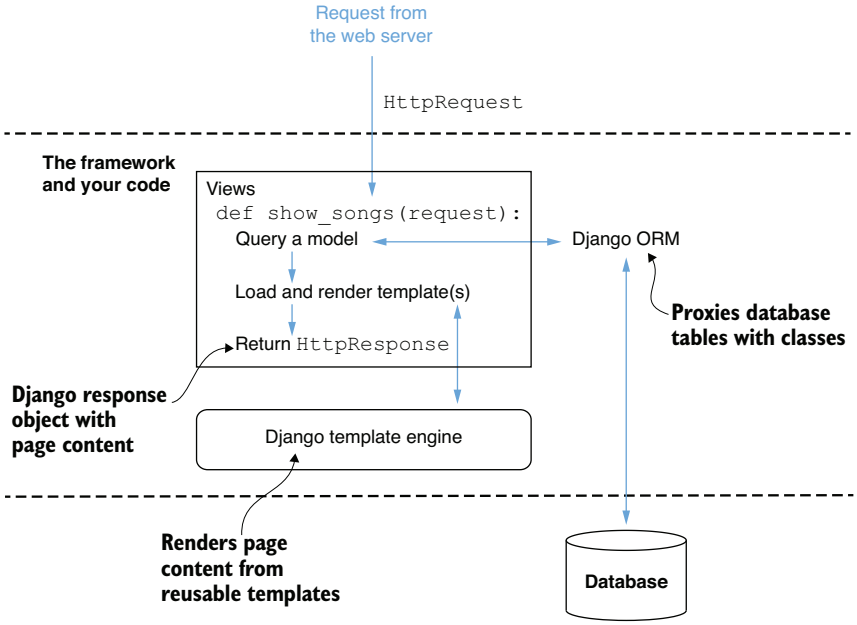


Figure 1.2 A view interacting with the database and template engine.

Consider a web page that shows the songs released in a given year. The URL for all the songs in 1942 might be http://example.com/song_by_year/1942/. The code for displaying this page starts with a mapping between the /song_by_year/ portion of the URL and a function to handle the response. The mapping is capable of parsing the /1942/ portion of the URL as a parameter and passing it into the view function that generates the page. Figure 1.3 shows the parts of the URL handled by the view and how the view interacts with the ORM.

The `song_by_year` view does a look-up in the database for all the songs in 1942. The database query gets abstracted through a `Song` class in the ORM, and a query gets performed by invoking a function on that class. The result from the database gets encapsulated in a `QuerySet` response object containing all the corresponding songs.

With the data in hand, the view creates the HTML response. It loads the songs.html template, which inherits from a common template that defines the look-